

Project Backlog — Unified Ticket Management System (UTMS)

Purpose

This document lists the **currently approved backlog** for the project — the explicit set of gaps, technical debt, and future feature enhancements reviewed and confirmed by the project owner. It supersedes any earlier, broader draft of this file: items that are not on this approved list have been removed from this document, not because they don't exist, but because they are out of scope for the current backlog cycle.

This is written for **whoever picks up this project next** — a new developer, a new lead, or the current team returning after time away.

This document does not replace the deeper technical docs — it points to them. For architectural detail, see:

- **docs/17-roadmap/** — original, more detailed roadmap files
- **docs/16-known-limitations/** — deliberate, accepted gaps (not bugs to fix)
- **docs/15-architecture-decisions/** — why the system is shaped the way it is
- Root **CLAUDE.md** — the dated engineering log this whole documentation set is grounded in

How to Read This Document

Priority

Level	Meaning
P0 — Critical	Actively breaks something the product promises (data loss, security hole, compliance guarantee). Fix before anything else.
P1 — High	Real, reproducible gap with a plausible near-term business, security, or support impact.
P2 — Medium	Real gap, but narrower blast radius, lower frequency, or only matters at a future scale.
P3 — Low	Low-risk, cosmetic, documentation-only, or a deliberately deferred item with no near-term pressure.

Status

Label	Meaning
Open	Confirmed gap, no fix scheduled.
Planned	A direction/design has been floated, but no code exists yet.
Deferred	A deliberate decision to not do this now — it's future work, not an active task.
Known Limitation	Working as designed today; listed so nobody re-discovers it as a surprise or "fixes" it without realizing it's intentional.

Snapshot — Approved Backlog Items

ID	Category	Title	Priority	Status
BL-01	A. Current Functional Gaps	Role Creation / Update / Deletion functionality not implemented	P1	Open
BL-04	A. Current Functional Gaps	No client/category scoped personalization for Staff and Team Lead	P2	Open
BL-08	A. Current Functional Gaps	Reporting Manager can view but not act on employees' escalated tickets	P2	Open
BL-09	A. Current Functional Gaps	No defined workflow once a ticket escalates beyond 3 times	P1	Open
BL-11	A. Current Functional Gaps	Self-claim and self-assignment are workflow rules, not permissions	P3	Known Limitation
BL-05	B. Security / Privacy	No PHI/PII detection anywhere in the application	P1	Open
BL-10	B. Security / Privacy	JWT auth relies on compensating mechanisms, not true session revocation	P1	Open
BL-02	C. Architecture / Technical Debt / Scalability	Shared mailbox ingestion uses polling, not webhooks	P3	Deferred
BL-03	C. Architecture / Technical Debt / Scalability	Notification email uses a shared inbox, not a dedicated mail service	P2	Deferred
BL-06	C. Architecture / Technical Debt / Scalability	APScheduler sweep may not be safe across multiple production instances	P2	Deferred
BL-07	C. Architecture / Technical Debt / Scalability	Attachment storage is on Supabase; future migration to S3	P3	Deferred
BL-12	D. Future Feature Enhancements	Workload-based ticket assignment	P2	Planned
BL-13	D. Future Feature Enhancements	Automated ticket creation	P3	Deferred

13 approved items. This document intentionally contains only these — no other backlog topics are in scope for this cycle.

A. Current Functional Gaps

BL-01 — Role Creation / Update / Deletion functionality not implemented

Current State The permissions **role:create**, **role:update**, and **role:delete** already exist in the RBAC permission catalog.

Problem / Gap This is **not** a permission problem — the permissions exist. The actual role-management functionality (creating a new role, updating an existing role's definition, deleting a role) has not been implemented yet.

Future Requirement Implement real role creation, role update, and role deletion functionality, gated by the existing **role:create** / **role:update** / **role:delete** permissions.

Priority: P1 — High. This is foundational RBAC administration capability that is currently entirely missing.

Acceptance Criteria

- A Super Admin (or any holder of the relevant permission) can create a new role.
- A Super Admin can update an existing role's name/description/attributes.
- A Super Admin can delete a role that is no longer needed.
- All three actions are gated by their corresponding existing permission.

Constraints / Notes

- Do not describe this as a permission issue — the permissions already exist and are correctly defined; only the functionality is missing.

BL-04 — No client/category scoped personalization for Staff and Team Lead

Current State Staff and Team Lead users currently have no proper client-scoped or category-scoped personalization in filters, client filters, category filters, the "From" selection in Compose, or related client/category mailbox selection behavior.

Problem / Gap Staff and Team Lead lack the scoped personalization that would let their filters and mailbox/compose selection reflect only the clients/categories relevant to them.

Future Requirement Implement appropriate client-scoped and category-scoped personalization for Staff and Team Lead users, covering filters, client filters, category filters, the "From" selection in Compose, and related mailbox selection behavior.

Priority: P2 — Medium. A real usability gap for these two roles, but not a security or data-integrity issue.

Acceptance Criteria

- Staff and Team Lead see filters, client filters, and category filters scoped appropriately to their own context.
- The "From" selection in Compose and related mailbox selection behavior reflects the same scoping for Staff and Team Lead.
- Account Manager and Site Lead behavior is verified unchanged after implementation (see Constraints).

Constraints / Notes

- **Existing Account Manager behavior must not change or be disturbed** — filters, client selection, category selection, "From" button, mailbox visibility, category visibility, and related functionality must remain exactly as they are today.
- **Existing Site Lead behavior must not change or be disturbed** — same list of behaviors as above.
- Do not describe Account Manager or Site Lead behavior as a problem — this item is scoped specifically to the Staff/Team Lead gap.

BL-08 — Reporting Manager can view but not act on employees' escalated tickets

Current State Reporting Managers can currently see their employees' tickets once those tickets are escalated.

Problem / Gap Reporting Managers have visibility only — they currently cannot perform any actions on those escalated tickets.

Future Requirement Define and implement appropriate action capabilities for Reporting Managers on their employees' escalated tickets.

Priority: P2 — Medium. A real gap in the Reporting Manager capability set, but the underlying visibility already works.

Acceptance Criteria

- A clear, agreed set of actions Reporting Managers may take on escalated tickets belonging to their employees is defined.
- Those defined actions are implemented and gated appropriately.

Constraints / Notes

- Do not invent or finalize which actions Reporting Managers should be allowed to perform — that set needs to be defined as part of this work, not assumed now.
-

BL-09 — No defined workflow once a ticket escalates beyond 3 times

Current State Tickets can currently escalate more than 3 times with no defined next step. The ticket continues notifying the Site Lead repeatedly, producing a notification loop.

Problem / Gap There is no clear workflow for what should happen after the third escalation — no defined next state, next action, or way to stop the repeated Site Lead notifications.

Future Requirement Define and implement a clear workflow for tickets that have escalated more than 3 times, addressing what happens after the third escalation, what the next state/action should be, and how repeated Site Lead notifications should be handled.

Priority: P1 — High. The current notification-loop behavior has a real, ongoing operational impact on the Site Lead.

Acceptance Criteria

- A defined next state/action exists for a ticket that has escalated more than 3 times.
- Repeated Site Lead notifications for the same escalation are handled according to the new, defined workflow.

Constraints / Notes

- Do not invent the final business workflow as part of this documentation update — only the current problem and the requirement to define one are documented here.
-

BL-11 — Self-claim and self-assignment are workflow rules, not permissions

Current State There are currently no dedicated permissions for (a) a user claiming a ticket for themselves from the Open Pool, or (b) a user being assigned their own ticket during ticket creation. Both behaviors are governed by workflow/business rules rather than a separate permission.

Problem / Gap None — this is documented as current, accurate behavior, not a defect.

Future Requirement None at this time. This item exists to record the current behavior accurately.

Priority: P3 — Low. Documentation-only; no functional gap identified.

Acceptance Criteria

- N/A — no implementation work is defined by this item.

Constraints / Notes

- Do not change this behavior.
 - Do not add a new permission for either action as part of this documentation update.
-

B. Security / Privacy

BL-05 — No PHI/PII detection anywhere in the application

Current State The application currently has no PHI/PII detection capability.

Problem / Gap Ticket content, emails, replies, notes, and attachments may contain PHI/PII with no detection mechanism in place today.

Future Requirement Implement PHI/PII detection as a future security/privacy capability, with ticket content, emails, replies, notes, and attachments as the areas to consider.

Priority: P1 — High. Given the healthcare-adjacent nature of the client data this system handles, this is a meaningful compliance/privacy exposure.

Acceptance Criteria

- A defined mechanism exists for detecting PHI/PII in at least the areas identified (ticket content, emails, replies, notes, attachments).
- The detection approach and its scope are documented before implementation begins.

Constraints / Notes

- Do not claim PHI/PII detection currently exists anywhere in the system.
 - Do not implement this now — this is a future requirement only.
-

BL-10 — JWT auth relies on compensating mechanisms, not true session revocation

Current State The application uses JWT-based authentication, with tokens/cookies stored in browser local storage rather than a traditional server-side HTTP session. A single shared secret key is used for JWT signing.

Problem / Gap Once issued, a JWT normally remains valid until expiration unless additional server-side checks intervene. If a user's access is revoked, or their permissions/roles change, an already-issued JWT can keep working until it expires. Today this is mitigated only by compensating mechanisms — permission-version checking, and a separate mechanism/table for impersonation sessions — not by true, immediate JWT revocation. Additionally, a single shared signing key means that if it is ever compromised, forged valid JWTs could potentially be created.

Future Requirement Redesign/improve the authentication/session architecture to address token/session revocation, permission/access changes, secure token/session storage, and JWT signing-key security.

Priority: P1 — High. This is a real security architecture gap, currently mitigated but not resolved.

Acceptance Criteria

- A defined approach exists for revoking an issued token/session immediately on access, permission, or role changes.
- Token/session storage and signing-key security are addressed as part of the same redesign.

Constraints / Notes

- Possible approaches may be evaluated later; do not prescribe or implement a specific solution now unless it is already documented in the project.

- Do not modify authentication as part of this backlog documentation task.
 - Current compensating mechanisms (permission-version checking, impersonation session table) remain in place and are not being removed or altered by this item.
-

C. Architecture / Technical Debt / Scalability

BL-02 — Shared mailbox ingestion uses polling, not webhooks

Current State Shared mailbox email ingestion currently uses polling.

Problem / Gap Polling is less scalable and less event-driven than a webhook-based approach would be.

Future Requirement Move from polling-based shared mailbox ingestion to webhook/event-driven processing in the future, to improve architecture scalability as mailbox processing needs grow.

Priority: P3 — Low. Current polling implementation works; this is a forward-looking architectural improvement, not an active problem.

Acceptance Criteria

- A webhook/event-driven ingestion path is designed and implemented as a replacement for polling.
- Existing mailbox ingestion functionality is preserved through the transition.

Constraints / Notes

- Do not modify the current polling implementation as part of this item.
 - Do not implement webhooks now — this is a future backlog item only.
-

BL-03 — Notification email uses a shared inbox, not a dedicated mail service

Current State System notifications are currently sent using **ticketing@probeps.com**, a shared inbox rather than a separate dedicated mail service.

Problem / Gap Using a shared inbox for system-generated notifications is less isolated/manageable than a dedicated mail service would be.

Future Requirement Use a separate/dedicated mail service for system-generated notifications instead of the shared inbox.

Priority: P2 — Medium. Not blocking today, but a real architectural improvement for notification reliability and manageability.

Acceptance Criteria

- A dedicated mail service is identified and used for system-generated notification email.
- Existing notification delivery behavior is preserved through the transition.

Constraints / Notes

- Current notifications continue to use **ticketing@probeps.com** — this is not being changed now.
 - Do not implement this now.
 - Do not choose a specific provider unless it already exists in the project.
-

BL-06 — APScheduler sweep may not be safe across multiple production instances

Current State The application currently uses Python APScheduler, running in-process, for sweep/scheduled processing (e.g. the SLA sweep).

Problem / Gap If the application is ever deployed with multiple production instances, APScheduler-based processing could result in the same scheduled work being executed redundantly by more than one instance.

Future Requirement Replace or redesign the sweep/scheduled processing architecture so that it is safe for a multi-instance production environment.

Priority: P2 — Medium. Only becomes a real problem if/when the deployment moves to multiple instances; the current single-instance deployment is not affected.

Acceptance Criteria

- A defined approach exists for running sweep/scheduled processing safely across multiple production instances (e.g. exactly-once execution per tick).
- The existing single-instance behavior is preserved until the redesign is implemented.

Constraints / Notes

- Do not remove APScheduler now.
 - Do not implement a replacement now.
 - Do not claim the current single-instance implementation is already broken — it works correctly today; this is a future scalability concern only.
-

BL-07 — Attachment storage is on Supabase; future migration to S3

Current State Attachments are currently stored using Supabase storage.

Problem / Gap None identified with current functionality — this is a planned architectural migration, not a defect.

Future Requirement Migrate attachment storage from Supabase to Amazon S3, accounting for existing attachment storage and current attachment functionality.

Priority: P3 — Low. Current storage works; this is a future infrastructure migration.

Acceptance Criteria

- A migration plan exists that accounts for existing stored attachments and current attachment functionality (upload, download, delete, external/linked attachments).
- Existing attachment functionality is preserved through the migration.

Constraints / Notes

- Do not perform the migration now.
 - Do not modify attachment storage code as part of this backlog update.
-

D. Future Feature Enhancements

BL-12 — Workload-based ticket assignment

Current State There is currently no workload-based ticket assignment mechanism. Assignable-candidate lists are filtered by role/category/hierarchy but not ranked or scored by workload.

Problem / Gap Supervisors have no system signal indicating that one agent already carries more open work than another when assigning or transferring tickets.

Future Requirement Implement workload-based assignment using relevant workload variables — current active tickets, existing workload, leave status, availability, and other relevant workload factors — to support better distribution of tickets.

Priority: P2 — Medium. A meaningful supervisor-experience improvement, not an active defect.

Acceptance Criteria

- Assignable-candidate lists can be scored/ranked using at least the identified workload variables.
- Existing assignment behavior remains available where workload ranking is not used.

Constraints / Notes

- Do not define the final scoring algorithm as part of this documentation update.
 - Do not implement this now.
-

BL-13 — Automated ticket creation

Current State Ticket creation today is primarily manual.

Problem / Gap Manual ticket creation is required even for workflows where an appropriate inbound/system source already exists in the application.

Future Requirement Automate ticket creation where appropriate, instead of requiring manual creation for every applicable workflow, using appropriate inbound/system sources already present in the application.

Priority: P3 — Low. A future efficiency enhancement, not a current defect.

Acceptance Criteria

- At least one appropriate inbound/system source is identified as a candidate for automated ticket creation.
- Automated creation preserves existing manual-creation functionality as a fallback/alternative.

Constraints / Notes

- Do not assume or invent final automation rules as part of this documentation update.
 - Do not implement this now.
-

Critical Preservation Requirements (Summary)

These facts must not be contradicted by any future work derived from this backlog:

- **Account Manager:** current behavior (filters, client selection, category selection, "From" button, mailbox visibility, category visibility, related functionality) must remain unchanged. See BL-04.
 - **Site Lead:** current behavior (same list as above) must remain unchanged. See BL-04.
 - **Shared Mailbox Polling:** polling remains the current implementation; webhooks are future-only. See BL-02.
 - **Notifications:** current notifications continue using **ticketing@probeps.com**; a dedicated mail service is future-only. See BL-03.
 - **Attachments:** current attachment storage remains on Supabase; S3 migration is future-only. See BL-07.
 - **Authentication:** current JWT authentication remains unchanged; the revocation/session architecture issue is technical debt for later. See BL-10.
 - **Self-Claim / Self-Assignment:** both remain workflow rules today; no new permission is introduced by this documentation update. See BL-11.
-

This document should be updated whenever an approved item here is resolved, re-scoped, or the approved backlog scope itself is revised by the project owner — treat it as a living checklist bounded by the currently approved scope, not a one-time snapshot.